

P2848R1

std::is_unique

Published Proposal, 2024-07-13

Authors:

[Arthur O'Dwyer](#)

[Enrico Mauro](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d2848-std-is-unique.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d2848-std-is-unique.html

Abstract

The STL provides the triples (`std::sort`, `std::is_sorted_until`, `std::is_sorted`) and (`std::make_heap`, `std::is_heap_until`, `std::is_heap`) and (`std::unique`, `std::adjacent_find`, [blank]). We fill in the blank by proposing `std::is_unique`.

Table of Contents

1 Changelog

2 Motivation and solution

2.1 Prior art and alternative names

2.2 Should we require `pred` to be an equivalence relation?

3 Implementation experience

4 Proposed wording relative to C++23

4.1 Header `<algorithm>` synopsis [algorithm.syn]

4.2 `is_uniqued` [alg.is.uniqued]

4.3 Header `<version>` synopsis [version.syn]

§ 1. Changelog

- R1 (post-St Louis 2024):
 - Minor wording nits (thanks Tim Song!)
 - Add feature test macro
 - Add SG9's naming ideas; also discuss the editorial section-naming question

§ 2. Motivation and solution

The "classic STL" provides the following sets of related algorithms:

- `is_sorted_until` (returns iterator), `is_sorted` (i.e. `is_sorted_until == end()`)
- `is_heap_until` (returns iterator), `is_heap` (i.e. `is_heap_until == end()`)
- `adjacent_find` (returns iterator), `[blank]` (i.e. `adjacent_find == end()`)
- `mismatch` (returns iterators), `equal` (i.e. `mismatch == {end(), end()}`)
- `find_if` (returns iterator), `none_of` (i.e. `find_if == end()`)
- `find` (returns iterator), `contains` (i.e. `find != end()`)
- `search` (returns iterator), `contains_subrange` (i.e. `search != end()`)

And these, too:

- `sort` (i.e. make the input range satisfy `is_sorted`)
- `make_heap` (i.e. make the input range satisfy `is_heap`)
- `unique` (i.e. make the input range satisfy `[blank]`)

The missing algorithm is what would fill in the blank in the following invariant assertions:

```
std::set<K> s = { ... };
std::multiset<K> ms = { ... };
std::map<K, V> m = { ... };
std::multimap<K, V> mm = { ... };
std::unordered_set<K> us = { ... };
std::unordered_map<K, V> um = { ... };

template<class M, class P = M::value_type>
auto ValueEq(const M& m) {
    return [eq = m.key_eq()](const P& a, const P& b) {
        return eq(a.first, b.first);
    };
}

assert(std::is_sorted(s.begin(), s.end(), s.value_comp()));
assert(std::is_sorted(ms.begin(), ms.end(), ms.value_comp()));
assert(std::is_sorted(m.begin(), m.end(), m.value_comp()));
assert(std::is_sorted(mm.begin(), mm.end(), mm.value_comp()));
assert(std::is____ed(s.begin(), s.end(), std::not_fn(s.value_comp())));
assert(std::is____ed(m.begin(), m.end(), std::not_fn(m.value_comp())));
assert(std::is____ed(us.begin(), us.end(), us.key_eq()));
assert(std::is____ed(um.begin(), um.end(), ValueEq(um)));
```

We propose that this algorithm should exist, and should be spelled `std::is_uniqued`.

§ 2.1. Prior art and alternative names

The C++ STL verb `std::unique` is derived from Unix's `uniq` filter, which filters adjacent matching lines from the input.

The Swift language provides a transformation named `sorted` (corresponding to C++'s `std::sort`), and a transformation named `uniqued`. The latter doesn't quite correspond to the C++/Unix notion of "uniquing": it removes even non-adjacent duplicates, by putting all the elements into a hashset and

then pulling them out again. Still, this is prior art for the idea of treating "unique, uniqued, uniquing" as a verb.

```
let animals = ["dog", "pig", "cat", "ox", "dog", "cat"]
let u = Array(animals.uniqued())
// 'u' is now ["dog", "pig", "cat", "ox"]
```

SG9 Ranges discussed P2848R0 in St Louis (June 2024). Two participants expressed discomfort with "is uniqued" as a verb form (although at least two others expressed that it was surely the correct name, given the names nearby it semantically). Alternatives proposed, without full discussion, were `is_unique` and `is_adjacent_unique`.

We believe that `is_unique(first, last)` is a dangerously wrong name, because the algorithm specifically does not check whether the range is in some sense "unique," and *certainly* does not check whether the range contains no duplicates. It checks specifically for *adjacent* duplicates, i.e., it checks that the range has had the mutating algorithm `unique` performed on it.

§ 2.2. Should we require `pred` to be an equivalence relation?

Currently, `std::unique` has undefined behavior if you pass it a predicate that is not an *equivalence relation*. Fortunately, `std::adjacent_find` does not.

For example, the author of `std::flat_set` might write:

```
container_type c;
key_compare compare;
std::sort(c.begin(), c.end(), compare);
c.erase(std::unique(c.begin(), c.end(), std::not_fn(compare)), c.end()); //
assert(std::is_sorted(c.begin(), c.end(), compare));
assert(std::adjacent_find(c.begin(), c.end(), std::not_fn(compare)) == c.e
```

Both before and after this proposal, the line marked "UB" has undefined behavior (although it will generally work in practice). We don't propose to change the precondition of `std::unique`. The line marked "OK" has well-defined behavior. we don't propose to change the precondition of `std::adjacent_find` either.

After this proposal, the following shorter line will be equivalent to the line marked "OK":

```
assert(std::is_uniqued(c.begin(), c.end(), std::not_fn(compare))); // OK
```

That is, the new algorithm `std::is_uniqued` will have the same precondition as the existing `std::adjacent_find` algorithm (in terms of which it is defined). This differs from the precondition of `std::unique`, which is a little surprising, but we believe it's the correct choice.

§ 3. Implementation experience

Arthur has implemented `is_uniqued` and `ranges::is_uniqued` in his fork of libc++; see [commit 490536e](#). This implementation is [available on Godbolt Compiler Explorer](#).

§ 4. Proposed wording relative to C++23

NOTE: The three other families are organized in umbrella sections: [alg.partitions] contains `is_partitioned`, `partition`, `partition_point`, and others, with no internal divisions. [alg.sort] is internally divided into [sort] `sort`, [is.sorted] `is_sorted` and `is_sorted_until`, and others. [alg.heap.operations] is internally divided into [make.heap] `make_heap`, [is.heap] `is_heap` and `is_heap_until`, and others. Should we therefore extract `adjacent_find` from [alg.nonmodifying]>[alg.adjacent.find] and `unique` from [alg.modifying.operations]>[alg.unique] into a new umbrella section [alg.uniquing] internally divided into [alg.unique], [alg.is.uniqued], and [alg.adjacent.find]? That is an editorial question to be decided by LWG during wording review. The proposed wording below doesn't do it (yet).

§ 4.1. Header `<algorithm>` synopsis [algorithm.syn]

Modify [\[algorithm.syn\]](#) as follows:

```

constexpr borrowed_iterator_t<R>
    adjacent_find(R&& r, Pred pred = {}, Proj proj = {});
}

// [alg.is.uniqued], is_uniqued
template<class ForwardIterator>
    constexpr bool is_uniqued(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class BinaryPredicate>
    constexpr bool is_uniqued(ForwardIterator first, ForwardIterator last,
                             BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator>
    bool is_uniqued(ExecutionPolicy&& exec, // see [algorithms
                  ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
    bool is_uniqued(ExecutionPolicy&& exec, // see [algorithms
                  ForwardIterator first, ForwardIterator last,
                  BinaryPredicate pred);
namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_binary_predicate<projected<I, Proj>,
                                     projected<I, Proj>> Pred = ranges::equal_to>
        constexpr bool is_uniqued(I first, S last, Pred pred = {}, Proj proj = {});
    template<forward_range R, class Proj = identity,
            indirect_binary_predicate<projected<iterator t<R>, Proj>,
                                     projected<iterator t<R>, Proj>> Pred = ranges::equ
        constexpr bool is_uniqued(R&& r, Pred pred = {}, Proj proj = {});
}

// [alg.count], count
template<class InputIterator, class T = iterator_traits<InputIterator>::value_type>
    constexpr typename iterator_traits<InputIterator>::difference_type
        count(InputIterator first, InputIterator last, const T& value);

```

§ 4.2. `is_uniqued` [alg.is.uniqued]

NOTE: This wording is copied straight from [\[is.sorted\]](#), with these mechanical replacements: `is_sorted` becomes `is_uniqued`, `is_sorted_until` becomes `adjacent_find`, `Compare comp` becomes `BinaryPredicate pred` (cf. [\[alg.adjacent.find\]](#)), `indirect_strict_weak_order` becomes `indirect_binary_predicate` (ditto), and `ranges::less` becomes `ranges::equal_to` (ditto).

Add this new section between [\[alg.adjacent.find\]](#) and [\[alg.count\]](#).

```
template<class ForwardIterator>
constexpr bool is_uniqued(ForwardIterator first, ForwardIterator last);
```

Returns: `adjacent_find(first, last) == last`.

```
template<class ExecutionPolicy, class ForwardIterator>
bool is_uniqued(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last);
```

Returns: `adjacent_find(std::forward<ExecutionPolicy>(exec), first, last) == last`.

```
template<class ForwardIterator, class BinaryPredicate>
constexpr bool is_uniqued(ForwardIterator first, ForwardIterator last,
                        BinaryPredicate pred);
```

Returns: `adjacent_find(first, last, pred) == last`.

```
template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
bool is_uniqued(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last,
               BinaryPredicate pred);
```

Returns: `adjacent_find(std::forward<ExecutionPolicy>(exec), first, last, pred) == last`.

```
template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_binary_predicate<projected<I, Proj>,
                                   projected<I, Proj>> Pred = ranges::equal_to>
constexpr bool ranges::is_uniqued(I first, S last, Pred pred = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_binary_predicate<projected<iterator_t<R>, Proj>,
                                   projected<iterator_t<R>, Proj>> Pred = ranges::equal_to>
constexpr bool ranges::is_uniqued(R&& r, Pred pred = {}, Proj proj = {});
```

Returns: `ranges::adjacent_find(first, last, pred, proj) == last`.

§ 4.3. Header `<version>` synopsis [version.syn]

Modify [version.syn] as follows:

```
#define __cpp_lib_is_swappable                201603L // freestanding, also in <tr>
#define __cpp_lib_is_uniqued                  20XXYYL // also in <algorithm>
#define __cpp_lib_is_within_lifetime          202306L // also in <type_traits>
```